

Micrograd: Neural Network Training From Scratch

Autograd Engine · Backpropagation · MLP Training

*“Neural networks are just mathematical expressions.
Backpropagation is just the chain rule, applied recursively.
Everything else is efficiency.”*

Author: Vatsav Reddy

Under the guidance of **Prof. Praveen Parchuri**

Lecture Reference: Andrej Karpathy — “Building Micrograd”

After reading these notes carefully, you will be able to:

- Read the full micrograd source and understand every line without hesitation
- Trace a forward pass and manually reconstruct what every intermediate `Value` holds
- Trace a backward pass and compute every `.grad` by hand
- Identify the exact closure that runs at each node and what it deposits
- Write your own custom differentiable operations with correct backward passes
- Spot the zero-grad bug, the overwrite bug, and the sign bug immediately
- Understand why PyTorch’s API looks the way it does

Read linearly the first time. Every code block is annotated to build mental models, not just recall.

Contents

1	What Problem Are We Actually Solving?	3
1.1	The Stack	3
2	Derivatives — What the Numbers Actually Mean	3
2.1	The Definition, Read Carefully	3
2.2	Numerical Estimation — The Sanity Check	4
2.3	Partial Derivatives — Multiple Inputs	4
3	The Chain Rule — The Only Idea Backpropagation Uses	5
3.1	Manual Backpropagation Through a Graph	5
4	The Value Class — Line by Line	6
4.1	The Constructor	6
4.2	Addition — <code>__add__</code>	7
4.3	Multiplication — <code>__mul__</code>	8
4.4	Power — <code>__pow__</code>	9
4.5	ReLU — <code>relu()</code>	10
4.6	Derived Operations — No New Math	10
4.7	Tanh — An Optional Alternative Activation	11
5	The Backward Pass — Topological Order and Chain Rule	12
5.1	Why Order Matters	12
5.2	The <code>backward()</code> Method	12
6	The Bugs — What Goes Wrong and Why	13
6.1	Forgetting to Zero Gradients	13
6.2	The Overwrite Bug When a Node Is Used Twice	13
6.3	The Sign Bug in the Update	14
7	The Neural Network Library — <code>nn.py</code>	14
7.1	Module — The Base Class	14
7.2	Neuron — The Atom	14
7.3	Layer — A Row of Independent Neurons	15
7.4	MLP — The Full Architecture	16
8	The Training Loop — Everything Together	17
8.1	The Dataset and Loss	17
8.2	The Training Loop	18
9	Connecting to PyTorch	19
10	The Complete Annotated Source	20
10.1	<code>engine.py</code>	20
10.2	<code>nn.py</code>	22
11	Complete Reference — Sticky Points	24
12	Reading Code as a Mental Simulation	26
12.1	The Forward-Pass Trace	26
12.2	The Backward-Pass Trace	26

12.3 Identifying Bugs by Pattern	26
12.4 Writing New Operations	27
13 Mental Model — The Three Questions	27

1. What Problem Are We Actually Solving?

A neural network is a function. It takes numbers in and produces numbers out. You want it to produce *specific* numbers — the right answers. To get it there, you adjust its internal knobs (the weights) by tiny amounts, over and over. The question is: which direction should each knob turn, and by how much?

That question is exactly what a derivative answers. For a weight w and a loss L , $\frac{\partial L}{\partial w}$ tells you exactly how much the loss changes when w changes by a tiny amount. If you know this for every single weight in the network, you know how to adjust all of them in parallel to make the loss go down.

The problem is that a real network might have billions of weights, and the expression connecting them to the loss is enormous. You cannot write out the derivative by hand. You need an algorithm that can compute all these derivatives automatically, for any expression, no matter how complex.

That algorithm is **backpropagation**. The software machinery that implements it is an **autograd engine**. Micrograd is a minimal, readable implementation of exactly this — 100 lines of Python.

Sticky Point

Everything in micrograd exists to answer one question: given an output L and any weight w buried deep in the expression that produced L , what is $\frac{\partial L}{\partial w}$?

1.1 The Stack

Your application — training loop, data, predictions

MLP / Layer / Neuron — composable neural net modules

Value class — builds expression graph, stores `.grad`, runs backward

Calculus — derivative definition and chain rule

Each layer uses the one below it but knows nothing about the ones above. The `Value` class does not know what a neuron is. A neuron does not know what a training loop is. This separation is what makes the design clean and reusable.

2. Derivatives — What the Numbers Actually Mean

2.1 The Definition, Read Carefully

Math: Derivative as limit

$$\left. \frac{df}{dx} \right|_{x=a} = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$$

This formula is not just notation — read it as an instruction. Start at point a . Take a tiny step h in the positive x direction. Measure how much f changed. Divide by h to normalize. As h shrinks toward zero, this ratio converges to the derivative.

The derivative is not a property of f globally — it is a number at a specific point. It changes

from point to point. At $x = 3$, the derivative of $f(x) = 3x^2 - 4x + 5$ is 14. At $x = -3$, it is -22 . These are completely different numbers describing completely different local behavior.

Intuition

The derivative tells you three things simultaneously: **sign** (does f go up or down when you nudge x up?), **magnitude** (how fast?), and **locality** (only valid near this specific x , not globally). All three matter in training.

2.2 Numerical Estimation — The Sanity Check

Before any autograd machinery, you can estimate derivatives by literally doing what the definition says: pick a small h , evaluate f twice, divide.

```

1 def f(x):
2     return 3*x**2 - 4*x + 5
3
4 x = 3.0
5 h = 0.0001
6
7 # Rise over run - the definition, made concrete
8 slope = (f(x + h) - f(x)) / h
9 print(slope)    # 14.0002... (analytically: 6x - 4 = 14 at x=3)

```

Code Walk-Through: Why this is useful but not scalable

This works for one variable at a time. For a neural net with N parameters, you'd need N forward passes just to estimate the gradient once — astronomically slow. Backpropagation computes all N gradients in roughly the cost of *two* forward passes. That is why we need autograd.

Use numerical estimation to *verify* that your autograd gives correct answers. This is called a gradient check.

2.3 Partial Derivatives — Multiple Inputs

When a function has multiple inputs, $\frac{\partial d}{\partial a}$ asks: if only a changes and everything else is fixed, how does d respond?

```

1 # d = a*b + c
2 a, b, c = 2.0, -3.0, 10.0
3 d = a*b + c          # forward: d = 4.0
4
5 h = 0.0001
6
7 # Partial w.r.t. a: hold b and c fixed, nudge only a
8 d2 = (a+h)*b + c
9 print((d2 - d) / h)  # -3.0 - equals b (analytically: dd/da = b)
10
11 # Partial w.r.t. b: hold a and c fixed
12 d2 = a*(b+h) + c
13 print((d2 - d) / h)  # 2.0 - equals a (analytically: dd/db = a)
14
15 # Partial w.r.t. c: trivially 1.0 because d(a*b + c)/dc = 1
16 d2 = a*b + (c+h)
17 print((d2 - d) / h)  # 1.0

```

Pattern: Analytic partial derivatives for common ops

These are the local gradients you will encounter over and over:

- **Addition** $d = a + b$: $\frac{\partial d}{\partial a} = 1$, $\frac{\partial d}{\partial b} = 1$
- **Multiplication** $d = a \cdot b$: $\frac{\partial d}{\partial a} = b$, $\frac{\partial d}{\partial b} = a$
- **Power** $d = a^n$: $\frac{\partial d}{\partial a} = n \cdot a^{n-1}$
- **ReLU** $d = \max(0, a)$: $\frac{\partial d}{\partial a} = 1$ if $a > 0$, else 0

These are *local* derivatives — they describe only what a single node contributes. To get the full gradient flowing back to a leaf, you chain them with the chain rule.

3. The Chain Rule — The Only Idea Backpropagation Uses

Math: Chain Rule

If z depends on y and y depends on x :

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

This extends to any chain of dependencies. If L depends on d which depends on c , then $\frac{\partial L}{\partial c} = \frac{\partial L}{\partial d} \cdot \frac{\partial d}{\partial c}$.

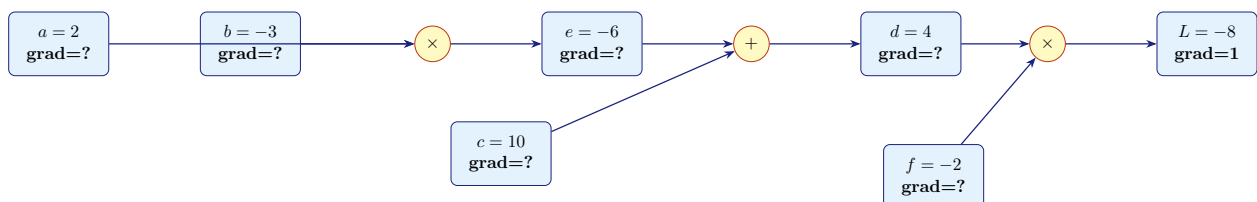
The multiplication is the key insight. You do not need to know the full path from c to L at once. You only need two pieces: how d influences L (which arrives as a gradient from upstream), and how c influences d (the local derivative, which the node can compute on its own).

Sticky Point

Every single node in the computation graph independently knows its own local derivative. It does not need to know anything about the rest of the graph. Backpropagation is just every node doing its local job and passing the result upstream. The chain rule says that multiplying these local contributions together gives the correct global gradient.

3.1 Manual Backpropagation Through a Graph

Consider $L = d \cdot f$ where $d = c + e$ and $e = a \cdot b$.



Step 1 — Base case: $\frac{\partial L}{\partial L} = 1$ always. **L.grad = 1.**

Step 2 — Multiply node $L = d \cdot f$: Local derivatives are $\frac{\partial L}{\partial d} = f = -2$ and $\frac{\partial L}{\partial f} = d = 4$. Chain rule deposits: **d.grad = 1 * (-2) = -2**, **f.grad = 1 * 4 = 4**.

Step 3 — Plus node $d = c + e$: Local derivative of $+$ is 1 for both inputs.

Chain rule: $c.\text{grad} = 1 * d.\text{grad} = -2$, $e.\text{grad} = 1 * d.\text{grad} = -2$.

Step 4 — Multiply node $e = a \cdot b$: Local derivative of e w.r.t. a is $b = -3$; w.r.t. b is $a = 2$.

Chain rule: $a.\text{grad} = (-3) * e.\text{grad} = (-3)(-2) = 6$, $b.\text{grad} = (2) * e.\text{grad} = (2)(-2) = -4$.

Note

$a.\text{grad} = 6$ means: if a increases by 0.001, then L increases by approximately $6 \times 0.001 = 0.006$. To make L decrease, you should decrease a . Gradient descent: $a.\text{data} -= \text{lr} * a.\text{grad}$.

Read This Like Code

Notice the pattern: every node needs only (a) its own local derivative and (b) the `.grad` arriving from its output side. It does not need to know the shape of the rest of the graph. This is why the backward pass can be implemented as a simple loop — each node is self-contained.

4. The Value Class — Line by Line

4.1 The Constructor

```

1 class Value:
2     def __init__(self, data, _children=(), _op=''):
3         self.data = data                # the number this node holds
4         self.grad = 0                   # dL/dself - filled in by backward
5                                         #
6         self._backward = lambda: None  # closure that propagates gradient
7                                         # set by ops
8         self._prev = set(_children)    # parent nodes that produced self
9         self._op = _op                  # which operation created this
10                                         # ('+', '*', etc.)

```

Code Walk-Through: Reading the constructor as a design

Every field exists for a specific purpose in the forward-backward cycle.

`self.data` is used during the forward pass. When you write $a * b$, Python multiplies `a.data` and `b.data` to produce the number in the output node.

`self.grad` starts at zero. The assumption encoded here is: “until proven otherwise, this node does not affect the output.” Backward accumulates the real gradient on top of this zero. If a node is never reached during backward (e.g., it is disconnected from the loss), its gradient legitimately stays zero.

`self._backward` is a Python closure. It is *defined* inside each arithmetic method and *captures* the variables from that method’s scope. When called later during the backward pass, it can still access `self`, `other`, and `out` even though those method calls have long since returned. This is the core mechanism by which each node knows how to propagate its gradient.

`self._prev` records which nodes were the inputs to this operation. Together, all `_prev` sets across all `Value` objects form the computation graph — a directed acyclic graph (DAG) where edges point from inputs to outputs.

`self._op` is purely for debugging and visualization. The math does not use it.

Pattern: The closure pattern — how `_backward` works

Every arithmetic operation follows this exact pattern:

1. Create the output `Value` with the correct forward value
2. Define a local function `_backward()` that closes over `self`, `other`, and `out`
3. Inside `_backward()`, deposit the chain rule result into parents using `+=`
4. Attach `_backward` to `out`
5. Return `out`

You will see this exact structure five times: in `__add__`, `__mul__`, `__pow__`, `relu`, and `tanh`. Recognizing the pattern lets you implement new operations without thinking from scratch.

4.2 Addition — `__add__`

```
1 def __add__(self, other):
2     # Guard: if other is a plain number, wrap it so it has a .data
3     attribute
4     other = other if isinstance(other, Value) else Value(other)
5
6     # Forward pass: create output node with correct data, record
7     children
8     out = Value(self.data + other.data, (self, other), '+')
9
10    # Backward closure: deposits chain rule gradient into both inputs
11    def _backward():
12        # Local derivative of + w.r.t. both inputs is 1
13        # Chain rule: input.grad += local_deriv * out.grad
14        self.grad += 1.0 * out.grad # simplifies to: self.grad += out
15        .grad
16        other.grad += 1.0 * out.grad
17
18    out._backward = _backward
19    return out
```


Code Walk-Through: Every decision in `__add__` explained

The isinstance guard: Writing `a + 2` where `a` is a `Value` and `2` is a plain Python int would crash without this. The guard converts the int into `Value(2)`, giving it a `.data` attribute. The same guard appears in `__mul__`. It is always the first line of any operation.

Why pass `(self, other)` as children: These are the inputs to this addition. Recording them here builds the graph incrementally — every time an operation runs, it connects itself to its inputs. By the time the forward pass is done, the full graph is in memory.

The closure captures by reference, not by value: When `_backward` is defined, Python does not copy `self`, `other`, and `out`. It stores references to those objects. When `_backward` is called later during `backward()`, it reads the *current* state of those objects — including any `.grad` values that have been updated since the forward pass.

Why `+=` not `=`: A `Value` node can be used more than once. Consider `b = a + a`. When this runs, `self` and `other` are the *same object* `a`. The `_backward` closure will be called once, and it will execute `self.grad += out.grad` and then `other.grad += out.grad` — but since `self` is `other`, both lines modify `a.grad`. Each adds `out.grad`, so the total is `2 * out.grad`. This is exactly correct by the multivariable chain rule. If we used `=` instead of `+=`, the second line would overwrite the first, and we would get the wrong answer.

Gradient of `+` is 1: If `out = self + other`, then $\frac{\partial \text{out}}{\partial \text{self}} = 1$ and $\frac{\partial \text{out}}{\partial \text{other}} = 1$ by the sum rule. Multiplying by `out.grad` (the chain rule factor) just gives `out.grad` back. A plus node is a **gradient forwarder** — it passes the upstream gradient through unchanged to all its inputs.

4.3 Multiplication — `__mul__`

```

1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), ' * ')
4
5     def _backward():
6         # Local derivative of out w.r.t. self is other.data (and vice
7           versa)
8         # Chain rule: multiply local derivative by incoming upstream
9           gradient
10        self.grad += other.data * out.grad
11        other.grad += self.data * out.grad
12
13    out._backward = _backward
14    return out

```

Code Walk-Through: The swap pattern in multiplication

This is perhaps the most important derivative to internalize. If `out = self * other`:

$$\frac{\partial \text{out}}{\partial \text{self}} = \text{other} \quad \frac{\partial \text{out}}{\partial \text{other}} = \text{self}$$

Each input's gradient involves *the other input's forward value*. This is why `_backward` reads `other.data` (the forward-pass value stored during the forward pass, not touched during backward) to compute `self`'s gradient. The forward values are frozen; only the `.grad` fields change during backward.

Think of it this way: if b is large, then changing a by a tiny amount has a big effect on $a \cdot b$ (the effect is proportional to b). So a gets a large gradient. If $b = 0$, then a has no effect — its gradient is zero. This intuition matches the formula exactly.

A multiply node is a **gradient router and scaler** — it routes the upstream gradient to each input, scaled by the other input's magnitude.

4.4 Power — `__pow__`

```

1 def __pow__(self, other):
2     # other must be a constant (int or float), not a Value
3     assert isinstance(other, (int, float)), "only supporting int/float
4         powers for now"
5
6     out = Value(self.data**other, (self,), f'**{other}')
7
8     def _backward():
9         # Power rule: d/dx [x^n] = n * x^(n-1)
10        # Here n = other, x = self.data
11        self.grad += (other * self.data**(other - 1)) * out.grad
12
13    out._backward = _backward
14    return out

```

Code Walk-Through: Why assert and why only constants

The `assert` is a design boundary. If `other` were a `Value` (a learnable exponent), the derivative with respect to `other` would involve logarithms: $\frac{\partial}{\partial n} x^n = x^n \ln x$. That requires a different, more complex backward. By restricting to constants, this implementation stays simple without sacrificing any practical use case — you almost never learn the exponent; you learn the base.

The power rule is the key building block because it enables division:

$$\frac{a}{b} = a \cdot b^{-1}$$

No new backward logic is needed. `a / b` becomes `a * (b ** -1)`, which reuses the already-defined `__mul__` and `__pow__` closures.

Sticky Point

The power of this design is that complex operations decompose into atoms you already have. Division reuses `__mul__` and `__pow__`. Subtraction reuses `__add__` and negation. Negation reuses `__mul__` with `-1`. You write the math naturally (`a / b`) and backpropagation works for free through the chain of simpler operations.

4.5 ReLU — `relu()`

```

1 def relu(self):
2     # Forward: pass through positives, clamp negatives to zero
3     out = Value(0 if self.data < 0 else self.data, (self,), 'ReLU')
4
5     def _backward():
6         # Derivative of ReLU: 1 when output > 0, else 0
7         # (out.data > 0) evaluates to True (=1) or False (=0) in Python
8         self.grad += (out.data > 0) * out.grad
9
10    out._backward = _backward
11    return out

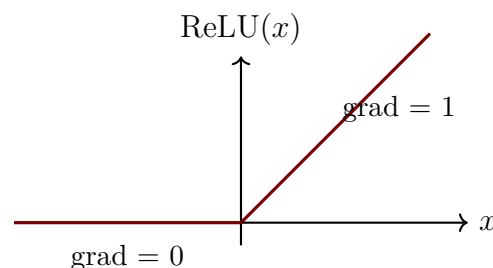
```

Code Walk-Through: The gate metaphor

`(out.data > 0)` is a boolean that Python treats as 1 or 0 in arithmetic. When the forward output was positive, the gate is open: gradient flows through unchanged. When the forward output was zero or negative, the gate is closed: gradient is blocked.

Notice the backward reads `out.data`, not `self.data`. Both would be equivalent here (if `self.data <= 0` then `out.data == 0`, and vice versa), but reading `out.data` is conceptually cleaner: it is the thing you computed, and you are asking whether it was positive.

The “dying ReLU” phenomenon follows directly from this code. A neuron whose output is perpetually negative will always have gradient zero — no signal ever reaches its weights, so they never update, so the neuron stays dead.

**4.6 Derived Operations — No New Math**

```

1 def __neg__(self):          return self * -1          # negate = multiply
   by -1
2 def __radd__(self, other):  return self + other      # fallback for: 2 +
   a
3 def __sub__(self, other):   return self + (-other)   # a - b = a + (-b)
4 def __rsub__(self, other):  return other + (-self)   # fallback for: 2 -
   a
5 def __rmul__(self, other):  return self * other      # fallback for: 2 *
   a
6 def __truediv__(self, other): return self * other**(-1) # a/b = a * b
   ^(-1)

```

```

7 def __rtruediv__(self, other): return other * self**-1 # fallback for:
  2 / a
8 def __repr__(self):
9     return f"Value(data={self.data}, grad={self.grad})"

```

Code Walk-Through: The `r*` fallback methods

Python operator resolution works like this: `2 * a` first asks `int` to multiply with a `Value`, which fails because `int` has no idea what a `Value` is. Python then falls back to `a.__rmul__`. Defining `__rmul__` as `return self * other` reverses the operands so the call becomes `a * 2`, which works.

Without these four `r*` methods, expressions like `1 - loss` (very common in loss functions), `2 * weight`, and `0.1 / param` would all crash with `TypeError`. They are not optional.

4.7 Tanh — An Optional Alternative Activation

```

1 import math
2
3 def tanh(self):
4     x = self.data
5     t = (math.exp(2*x) - 1) / (math.exp(2*x) + 1) # compute tanh(x)
6     out = Value(t, (self,), 'tanh')
7
8     def _backward():
9         # Derivative of tanh: d/dx tanh(x) = 1 - tanh(x)^2
10        # t is tanh(x), already computed in forward pass - reuse it here
11        self.grad += (1 - t**2) * out.grad
12
13    out._backward = _backward
14    return out

```

Code Walk-Through: Abstraction level is a design choice

You can implement `tanh` at two different levels of granularity:

Fine-grained: Implement `exp`, then build `tanh` from `exp`, `-`, `/`. The computation graph grows: it will have multiple nodes for each intermediate step. But every node is simple, and autograd handles the whole thing.

Coarse-grained: Implement `tanh` as one node using its known closed-form derivative $1 - t^2$. The graph has fewer nodes and may run slightly faster. The gradients are identical. Both approaches give exactly the same gradients at the leaves. Choosing coarse-grained makes sense when you have a reliable formula for the derivative. Choosing fine-grained makes sense when you want maximum flexibility and composability. Neither is wrong.

Math: tanh derivative

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

Since $t = \tanh(x)$ is computed during the forward pass and stored as a local variable, the backward reuses it: `self.grad += (1 - t**2) * out.grad`. At $x = 0$, the derivative is 1 (maximum). At large $|x|$, $\tanh \rightarrow \pm 1$ so the derivative $\rightarrow 0$ — this is the vanishing gradient problem.

5. The Backward Pass — Topological Order and Chain Rule

5.1 Why Order Matters

You cannot call `_backward()` on a node until every node that depends on it has already had its `_backward()` called. Consider $d = c + e$ and $L = d \cdot f$. To compute `c.grad`, you need `d.grad` to already be filled in. But `d.grad` is filled in when `L`'s `_backward()` runs. So you must process `L` before `d`, and `d` before `c` and `e`.

This ordering constraint is exactly what **topological sort** produces: an ordering of all nodes such that every node appears before its inputs.

5.2 The `backward()` Method

```

1 def backward(self):
2     # Phase 1: Build topological ordering of the entire computation
      graph
3     topo = []
4     visited = set()
5
6     def build_topo(v):
7         if v not in visited:
8             visited.add(v)
9             for child in v._prev:      # recurse into all inputs (children
              ) first
10                build_topo(child)
11                topo.append(v)          # only add self AFTER all inputs
              are done
12
13    build_topo(self)    # start from the output node (e.g. loss)
14
15    # Phase 2: Seed the root gradient
16    self.grad = 1      # dL/dL = 1 always - this is the base case
17
18    # Phase 3: Walk backward through the graph, applying each node's
      backward
19    for v in reversed(topo):    # reversed: root first, leaves last
20        v._backward()
```

Code Walk-Through: Dissecting `build_topo` recursively

Start at the root (say, `loss`). Mark it visited. Recurse into each of its `_prev` children. For each child, mark it visited and recurse into its children. A node is only appended to `topo` *after* all its children have been appended.

Result: `topo[0]` is a leaf (no parents — it was one of the first to have all children done, which for leaves is trivially true). `topo[-1]` is the root.

`reversed(topo)` starts at the root and ends at the leaves — exactly the order in which gradients must flow.

The `visited` set is essential. Without it, any node used more than once would be added to `topo` multiple times and have its `_backward()` called multiple times. The `+=` in each `_backward()` would cause gradients to accumulate incorrectly. The `visited` set ensures every node's backward runs exactly once.

Sticky Point

The complete cycle in one sentence: Forward pass builds the graph (by creating `Value` nodes and setting their `_backward` closures); `loss.backward()` topologically sorts the graph, sets `loss.grad=1`, and calls each `_backward()` in reverse order; afterward, every parameter's `.grad` holds the correct partial derivative of the loss with respect to that parameter.

6. The Bugs — What Goes Wrong and Why

6.1 Forgetting to Zero Gradients

Common Bug

The most common bug in neural network training. Every `_backward()` uses `+=`. If you call `backward()` twice without resetting `.grad` to zero between calls, the second backward adds on top of the first. Gradients accumulate indefinitely. Symptom: the network may seem to converge faster than expected (because the effective step size is inflated by accumulated gradients), then suddenly diverge, or converge to a wrong solution.

```
1 # WRONG: gradients from step 0 persist into step 1, 2, 3...
2 for step in range(100):
3     loss = compute_loss(model, xs, ys)
4     loss.backward()           # += on already-nonzero grads
5     update(model)
6
7 # CORRECT: reset before every backward
8 for step in range(100):
9     loss = compute_loss(model, xs, ys)
10    for p in model.parameters():
11        p.grad = 0           # reset to zero
12    loss.backward()          # now accumulates from a clean slate
13    update(model)
```

6.2 The Overwrite Bug When a Node Is Used Twice

```
1 # Wrong implementation using = instead of +=
2 def _backward():
3     self.grad = other.data * out.grad  # BUG: overwrites, not
4     other.grad = self.data * out.grad  # accumulates
5
6 # Bug manifests here:
7 a = Value(3.0)
8 b = a * a          # a is used as both self AND other
9 b.backward()
10 print(a.grad)     # should be 2*3 = 6, but wrong impl gives only 3
```

When `a * a` runs, `self` and `other` both point to the same object `a`. With `=`, the second assignment overwrites the first. With `+=`, both assignments accumulate, giving the correct result: $2a = 6$. The `+=` in `_backward()` is not optional; it is the fix to this fundamental issue.

6.3 The Sign Bug in the Update

```

1 # WRONG: increases the loss
2 p.data += lr * p.grad # + sign goes uphill (maximizes L)
3
4 # CORRECT: decreases the loss
5 p.data -= lr * p.grad # - sign goes downhill (minimizes L)

```

The gradient points in the direction that *increases* the loss. Gradient *descent* moves in the opposite direction. The minus sign is the descent. If you get the sign wrong, your network will run gradient *ascent* — the loss will go up, your predictions will get worse, and everything will diverge.

7. The Neural Network Library — nn.py

7.1 Module — The Base Class

```

1 class Module:
2     def zero_grad(self):
3         for p in self.parameters():
4             p.grad = 0 # reset before each backward pass
5
6     def parameters(self):
7         return [] # subclasses override to return their
                   # trainable params

```

Code Walk-Through: Why have a base class at all

`Module` exists so that every object in the hierarchy — `Neuron`, `Layer`, `MLP` — has the same interface. You can call `model.zero_grad()` and `model.parameters()` on any of them, regardless of internal complexity.

This mirrors `torch.nn.Module` in PyTorch exactly. The design choice is deliberate: learning micrograd gives you the mental model for PyTorch's module system.

The default `parameters()` returns `[]`, encoding the assumption that a bare `Module` has nothing trainable. Subclasses override this to return their actual `Value` objects.

7.2 Neuron — The Atom

```

1 class Neuron(Module):
2
3     def __init__(self, nin, nonlin=True):
4         # nin: number of inputs to this neuron
5         self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
6         self.b = Value(0) # bias: starts at zero
7         self.nonlin = nonlin # whether to apply ReLU after the
                             # linear combination
8
9     def __call__(self, x):
10        # Compute: w*x + b (dot product of weights and inputs, plus
11        # bias)
12        # sum(..., start) starts the accumulation from self.b (a Value)
13        # This ensures the entire sum is a Value object, not a plain
14        # float
15        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
16        return act.relu() if self.nonlin else act

```



```

15
16     def parameters(self):
17         return self.w + [self.b]      # nin weights + 1 bias = nin+1 params
18                                     total
19
20     def __repr__(self):
21         return f"{'ReLU' if self.nonlin else 'Linear'}Neuron({len(self.w)
22                                     })"

```

Code Walk-Through: Every decision in Neuron

Random initialization: Weights are sampled from $\text{Uniform}(-1, 1)$. If all weights started at zero, every neuron in a layer would compute the same output and receive the same gradient — they would forever remain identical. Random initialization breaks this symmetry.

Bias starts at zero: The bias shifts the activation function left or right. Zero is a neutral starting point — the network learns the right offset during training.

`sum(..., self.b)` — the start parameter: Python's built-in `sum(iterable, start=0)` starts accumulating from `start`. If you leave the default `start=0` (a plain Python int), the first addition would be `int + Value`, which triggers `int.__add__(Value)`, which fails. By passing `self.b` (a `Value`) as the start, all arithmetic stays within the `Value` universe. Every intermediate result remains a `Value`, and every operation is tracked in the graph.

`zip(self.w, x)`: Pairs each weight with the corresponding input: `[(w0,x0), (w1,x1), ...]`. The generator expression `(wi * xi for wi, xi in zip(self.w, x))` produces the products lazily, and `sum` accumulates them.

`nonlin=False` for the output layer: The output should be raw numbers (logits) so the loss function receives unrestricted values. Applying ReLU to the output would clamp negatives to zero, breaking the gradient signal. Always pass `nonlin=False` to the last layer.

What `__call__` does: When you write `neuron(x)`, Python internally calls `neuron.__call__(x)`. This means neurons are callable objects — they look and behave like functions while being full objects with learnable state.

Math: The neuron's computation

$$\text{output} = \text{ReLU}\left(\sum_{i=1}^n w_i x_i + b\right)$$

The sum $\sum w_i x_i + b$ is the linear combination. The ReLU is the nonlinearity. Without the nonlinearity, stacking multiple layers would be equivalent to a single layer (a composition of linear functions is linear). The nonlinearity is what makes deep networks expressively more powerful than shallow ones.

7.3 Layer — A Row of Independent Neurons

```

1 class Layer(Module):
2
3     def __init__(self, nin, nout, **kwargs):
4         # nin: input dimension for each neuron
5         # nout: how many neurons in this layer
6         # **kwargs: passed straight through to Neuron (e.g. nonlin=False)
7         self.neurons = [Neuron(nin, **kwargs) for _ in range(nout)]

```



```

8
9     def __call__(self, x):
10         # All neurons see the same input x, compute independently
11         out = [n(x) for n in self.neurons]
12         # Convenience: if there is only one output, unwrap from list
13         return out[0] if len(out) == 1 else out
14
15     def parameters(self):
16         # Flatten: for each neuron, collect its parameters; combine all
17         return [p for n in self.neurons for p in n.parameters()]
18
19     def __repr__(self):
20         return f"Layer of [{', '.join(str(n) for n in self.neurons)}]"

```

Code Walk-Through: Layer design decisions

All neurons see the same input: This is the definition of a fully-connected (dense) layer. Input x is broadcast to every neuron. Each neuron has its own independent weights and bias — they are not shared.

Outputs are computed independently: In a real library, this would happen in parallel (vectorized over a matrix). In micrograd, it is a list comprehension — conceptually equivalent but sequential.

****kwargs pass-through:** The keyword argument `nonlin=False` needs to reach the individual neurons of the last layer. By using ****kwargs**, `Layer` can accept arbitrary extra arguments and forward them to each `Neuron` constructor without knowing what those arguments mean.

Flattened parameters(): The nested list comprehension `[p for n in self.neurons for p in n.parameters()]` reads as: for each neuron, get its parameter list; pool all parameter lists into a single flat list. This lets `model.parameters()` at the top level return everything as one flat iterable.

Single-output unwrap: The output layer typically has exactly one neuron (for binary classification or regression). In that case, returning a list of one element would be awkward. The `return out[0] if len(out) == 1 else out` handles this gracefully.

7.4 MLP — The Full Architecture

```

1 class MLP(Module):
2
3     def __init__(self, nin, nouts):
4         # nin: number of input features (e.g., 3 for a 3D input)
5         # nouts: list of layer sizes [hidden1, hidden2, ..., output]
6         # Example: MLP(3, [4, 4, 1]) means 3 inputs -> 4 -> 4 -> 1
7             output
8
9         sz = [nin] + nouts    # e.g., [3, 4, 4, 1]
10
11        # Create layers from consecutive pairs of sizes
12        # Layer(3,4): first layer  Layer(4,4): second  Layer(4,1):
13            output
14        self.layers = [
15            Layer(sz[i], sz[i+1], nonlin=(i != len(nouts)-1))
16            for i in range(len(nouts))
17        ]
18        # nonlin=True for all layers except the last one (i != len(nouts)-1)

```

```

17
18     def __call__(self, x):
19         # Feed-forward: output of each layer becomes input of the next
20         for layer in self.layers:
21             x = layer(x)
22         return x
23
24     def parameters(self):
25         return [p for layer in self.layers for p in layer.parameters()]
26
27     def __repr__(self):
28         return f"MLP of [{', '.join(str(layer) for layer in self.layers)}]"

```

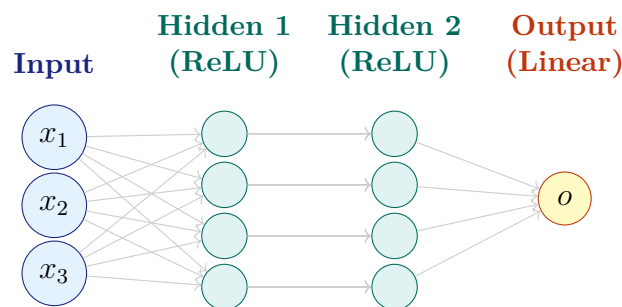
Code Walk-Through: The sz trick and the nonlin condition

sz = [nin] + nouts: This prepends the input dimension to the list of layer sizes. For `MLP(3, [4, 4, 1])`: `sz = [3, 4, 4, 1]`. Then `sz[i]` and `sz[i+1]` form consecutive pairs: (3,4), (4,4), (4,1) — exactly the input and output size of each successive layer.

nonlin=(i != len(nouts)-1): This boolean expression is `True` for every layer except the last one. `len(nouts)-1` is the index of the last layer. At that index, `i == len(nouts)-1`, so the expression is `False`, disabling the nonlinearity. Every hidden layer gets ReLU; the output layer does not.

Sequential `__call__`: The loop `for layer in self.layers: x = layer(x)` is the entire forward pass. The output of layer i replaces `x` and becomes the input to layer $i + 1$. After the loop, `x` is the final output.

Flat `parameters()`: The same nested list comprehension pattern as in `Layer`, but one level deeper. For `MLP(3, [4, 4, 1])`: 3 layers, each with several neurons, each with several parameters. All of them end up in one flat list — 41 total for this configuration.



8. The Training Loop — Everything Together

8.1 The Dataset and Loss

```

1  # 4 training examples, 3 features each
2  xs = [
3      [2.0, 3.0, -1.0],
4      [3.0, -1.0, 0.5],
5      [0.5, 1.0, 1.0],
6      [1.0, 1.0, -1.0],
7  ]
8  ys = [1.0, -1.0, -1.0, 1.0] # binary targets: +1 or -1
9
10 model = MLP(3, [4, 4, 1])

```

```

11 print(f"Parameters: {len(model.parameters())}") # 41

1 # Forward pass: get predictions for all examples
2 ypred = [model(x) for x in xs]
3
4 # Mean Squared Error loss: each term is a Value object
5 # Because ypred contains Value objects, every arithmetic op here is
   tracked
6 loss = sum((yout - ygt)**2 for ygt, yout in zip(ys, ypred))
7
8 # loss is now a single Value at the tip of a massive computation graph
9 # Calling loss.backward() will propagate gradients through the entire
   graph
10 print(loss.data) # e.g., 7.12

```

Math: Mean Squared Error

$$L = \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

- $\hat{y}_i$: model prediction for example i (a `Value` object)
- y_i : ground truth target (a plain Python float)
- When `yout - ygt` runs: `ygt` is a float, so Python tries `float.__sub__(Value)`, fails, then falls back to `Value.__rsub__(float)`, which succeeds
- The result is a `Value`; squaring it via `**2` creates another `Value`; summing all creates the final `loss Value`

The loss is zero only when every prediction exactly matches its target. Any mismatch produces a positive contribution. Minimizing L pushes all predictions toward their targets simultaneously.

8.2 The Training Loop

```

1 model = MLP(3, [4, 4, 1])
2 learning_rate = 0.05
3
4 for step in range(50):
5
6     # -- 1. FORWARD PASS
       -----
7     # Run all examples through the model
8     # Each model(x) call builds a computation subgraph
9     # All subgraphs share the same weight nodes
10    ypred = [model(x) for x in xs]
11    loss = sum((yout - ygt)**2 for ygt, yout in zip(ys, ypred))
12
13    # -- 2. ZERO GRADIENTS
       -----
14    # MUST come before backward, not after
15    # All _backward() use +=, so old gradients would corrupt new ones
16    for p in model.parameters():
17        p.grad = 0
18
19    # -- 3. BACKWARD PASS
       -----
20    # Topological sort + chain rule

```

```

21     # After this, every p in model.parameters() has a correct p.grad
22     loss.backward()
23
24     # -- 4. GRADIENT DESCENT UPDATE
25     -----
26     # Subtract: go against the gradient to minimize the loss
27     # The - sign is not optional: + would maximize the loss
28     for p in model.parameters():
29         p.data -= learning_rate * p.grad
30
31     if step % 10 == 0:
32         preds_str = [f"{y.data:.3f}" for y in ypred]
33         print(f"step {step:3d} | loss = {loss.data:.6f} | preds = {
34             preds_str}")

```

Code Walk-Through: Reading the training loop as a rhythm

There are exactly four steps, in exactly this order, on every iteration:

Forward: Run inputs through the model. A new computation graph is built each time. The weights (`p.data`) are the shared nodes that persist across iterations. The intermediate nodes (activations) are freshly created each step.

Zero grad: Reset all `.grad` fields to zero. This must happen before `backward()`, not after. If you do it after, the gradients sit at zero throughout the entire update step, which is actually fine — but then you lose the gradient information before you’ve used it. Standard practice: zero first, then backward, then update.

Backward: `loss.backward()` runs the topological sort on the current graph and calls every `_backward()` closure in reverse order. After this call returns, every parameter’s `.grad` holds exactly $\frac{\partial L}{\partial p}$ for this batch of examples and this configuration of weights.

Update: `p.data -= lr * p.grad`. The learning rate `lr` controls the step size. Too large: you overshoot and the loss explodes. Too small: convergence is very slow. The minus sign makes the update go downhill.

Math: Gradient Descent

$$p \leftarrow p - \eta \cdot \frac{\partial L}{\partial p}$$

η (eta) is the learning rate. The gradient $\frac{\partial L}{\partial p}$ points in the direction of steepest *increase* of the loss. Subtracting it moves p in the direction of steepest decrease — downhill on the loss surface.

Warning

Learning rate sensitivity. If `lr = 0.5` causes the loss to oscillate or grow, try `lr = 0.05` or `lr = 0.01`. A typical sign of too-large LR is the loss bouncing up and down each step. A typical sign of too-small LR is the loss barely decreasing after many steps.

9. Connecting to PyTorch

```

1 import torch
2
3 # Scalar values as 1-element tensors; .double() for float64 (like Python
4   floats)
5 x1 = torch.Tensor([2.0]).double(); x1.requires_grad = True

```

```

5 x2 = torch.Tensor([0.0]).double(); x2.requires_grad = True
6 w1 = torch.Tensor([-3.0]).double(); w1.requires_grad = True
7 w2 = torch.Tensor([1.0]).double(); w2.requires_grad = True
8 b = torch.Tensor([6.8813735870195432]).double(); b.requires_grad = True
9
10 n = x1*w1 + x2*w2 + b
11 o = torch.tanh(n)           # PyTorch has tanh built in
12
13 o.backward()                # same call as micrograd
14
15 # .item() extracts the scalar from a single-element tensor
16 print(o.data.item())        # 0.7071 - same as micrograd
17 print(x2.grad.item())       # 0.5
18 print(w2.grad.item())       # 0.0
19 print(x1.grad.item())       # -1.5
20 print(w1.grad.item())       # 1.0
21 # All identical to micrograd results

```

Tip

The differences between micrograd and PyTorch:

Tensors vs scalars: PyTorch wraps n -dimensional arrays; micrograd wraps single numbers. The math is the same. `.item()` extracts one number from a tensor.

`requires_grad=False` by default: PyTorch optimizes by not tracking gradients unless you ask. Leaf nodes that are inputs (data, not parameters) usually don't need gradients. Only parameters get `requires_grad=True`.

`float32` vs `float64`: PyTorch defaults to 32-bit floats for speed on GPUs. Python uses 64-bit floats. Cast with `.double()` to match.

API compatibility: `.backward()`, `.grad`, `.data`, `torch.nn.Module`, `.zero_grad()`, `.parameters()` — all these exist in both micrograd and PyTorch, with identical semantics. Learning micrograd means you already know the PyTorch API conceptually.

10. The Complete Annotated Source

10.1 engine.py

```

1 class Value:
2     """
3     A node in the computation graph.
4     Wraps one scalar, tracks how it was produced, and stores its
5     gradient.
6     """
7
8     def __init__(self, data, _children=(), _op=''):
9         self.data = data                # the scalar value
10        self.grad = 0                    # dL/dself; filled by
11        backward()                      # closure set by each op;
12        self._backward = lambda: None    # default no-op
13        self._prev = set(_children)      # set of input nodes that
14        produced self                    # string label for
15        self._op = _op                   debugging

```

```

14 # -- FORWARD OPERATIONS + BACKWARD REGISTRATION
15 -----
16 def __add__(self, other):
17     other = other if isinstance(other, Value) else Value(other)
18     out = Value(self.data + other.data, (self, other), '+')
19
20     def _backward():
21         # d(self + other)/d(self) = 1 -> self.grad += 1 * out.
22         # d(self + other)/d(other) = 1 -> other.grad += 1 * out.
23         self.grad += out.grad
24         other.grad += out.grad
25
26     out._backward = _backward
27     return out
28
29 def __mul__(self, other):
30     other = other if isinstance(other, Value) else Value(other)
31     out = Value(self.data * other.data, (self, other), '*')
32
33     def _backward():
34         # d(self * other)/d(self) = other -> self.grad += other.
35         # d(self * other)/d(other) = self -> other.grad += self.
36         self.grad += other.data * out.grad
37         other.grad += self.data * out.grad
38
39     out._backward = _backward
40     return out
41
42 def __pow__(self, other):
43     assert isinstance(other, (int, float)) # exponent must be a
44     # constant
45     out = Value(self.data**other, (self,), f'**{other}')
46
47     def _backward():
48         # Power rule: d/dx [x^n] = n * x^(n-1)
49         self.grad += (other * self.data**(other - 1)) * out.grad
50
51     out._backward = _backward
52     return out
53
54 def relu(self):
55     out = Value(0 if self.data < 0 else self.data, (self,), 'ReLU')
56
57     def _backward():
58         # Derivative: 1 if output > 0, else 0
59         # (out.data > 0) is True (=1) or False (=0)
60         self.grad += (out.data > 0) * out.grad
61
62     out._backward = _backward
63     return out
64
65 # -- TOPOLOGICAL SORT + CHAIN RULE APPLICATION
66 -----
67 def backward(self):

```

```

67     topo, visited = [], set()
68
69     def build_topo(v):
70         if v not in visited:
71             visited.add(v)
72             for child in v._prev:
73                 build_topo(child)
74             topo.append(v)    # self added only after all inputs are
                               # done
75
76     build_topo(self)
77
78     self.grad = 1            # base case: dL/dL = 1
79     for v in reversed(topo):
80         v._backward()        # each node deposits gradient into its
                               # inputs
81
82     # -- DERIVED OPERATIONS - reuse existing backward logic
83     -----
84
85     def __neg__(self):        return self * -1
86     def __radd__(self, other): return self + other    # handles: 2 +
87     def __sub__(self, other): return self + (-other)
88     def __rsub__(self, other): return other + (-self) # handles: 2 -
89     def __rmul__(self, other): return self * other    # handles: 2 *
90     def __truediv__(self, other): return self * other**-1
91     def __rtruediv__(self, other): return other * self**-1 # handles:
92     2 / a
93
94     def __repr__(self):
95         return f"Value(data={self.data}, grad={self.grad})"

```

10.2 nn.py

```

1  import random
2  from micrograd.engine import Value
3
4
5  class Module:
6      """Base class for all neural network components."""
7
8      def zero_grad(self):
9          for p in self.parameters():
10             p.grad = 0    # reset gradient before each backward pass
11
12     def parameters(self):
13         return []         # subclasses override to return their Value
14                             # objects
15
16     class Neuron(Module):
17         """
18         Single artificial neuron.
19         Computes: ReLU(w*x + b) or w*x + b (if nonlin=False)
20         """
21

```

```

22     def __init__(self, nin, nonlin=True):
23         # nin weights, one per input dimension
24         self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
25         self.b = Value(0) # bias initialized to zero
26         self.nonlin = nonlin # controls whether ReLU is applied
27
28     def __call__(self, x):
29         # sum(..., start=self.b): accumulates wi*xi values into a Value
30         # Starting from self.b (a Value) ensures the result stays a
31         # Value
32         act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
33         return act.relu() if self.nonlin else act
34
35     def parameters(self):
36         return self.w + [self.b] # nin+1 trainable scalars
37
38     def __repr__(self):
39         return f"{'ReLU' if self.nonlin else 'Linear'}Neuron({len(self.w)})"
40
41 class Layer(Module):
42     """
43     A fully-connected layer: nout independent neurons, each seeing the
44     same input.
45     """
46
47     def __init__(self, nin, nout, **kwargs):
48         self.neurons = [Neuron(nin, **kwargs) for _ in range(nout)]
49
50     def __call__(self, x):
51         out = [n(x) for n in self.neurons]
52         return out[0] if len(out) == 1 else out # unwrap single-neuron
53         layers
54
55     def parameters(self):
56         # Flatten: all parameters from all neurons into one list
57         return [p for n in self.neurons for p in n.parameters()]
58
59     def __repr__(self):
60         return f"Layer of [{', '.join(str(n) for n in self.neurons)}]"
61
62 class MLP(Module):
63     """
64     Multi-Layer Perceptron: a sequence of fully-connected layers.
65     Each layer's output feeds the next layer's input.
66     """
67
68     def __init__(self, nin, nouts):
69         sz = [nin] + nouts # e.g., MLP(3, [4,4,1]) -> sz = [3, 4, 4, 1]
70         self.layers = [
71             Layer(sz[i], sz[i+1], nonlin=(i != len(nouts)-1))
72             for i in range(len(nouts))
73         ]
74         # nonlin is True for hidden layers, False only for the last
75         # layer
76
77     def __call__(self, x):
78         for layer in self.layers:

```



```

77         x = layer(x)    # feed-forward: output of layer i -> input of
                          # layer i+1
78     return x
79
80     def parameters(self):
81         return [p for layer in self.layers for p in layer.parameters()]
82
83     def __repr__(self):
84         return f"MLP of [{', '.join(str(layer) for layer in self.layers)}]"

```

11. Complete Reference — Sticky Points

Sticky Point

1. What a Value holds.

`.data`: the number. `.grad`: $\partial L / \partial \text{self}$. `._backward`: closure for gradient propagation. `._prev`: input nodes. `._op`: label.

Sticky Point

2. `.grad` starts at 0.

Encodes “no effect assumed.” Backward accumulates via `+=` to fill in the real value. If a node is unreachable, grad stays 0 — correctly.

Sticky Point

3. Always `+=` in backward.

Multiple uses of the same node each contribute a gradient. `+=` accumulates all contributions. `=` would overwrite them — wrong.

Sticky Point

4. Closures capture references.

`._backward` captures `self`, `other`, `out` by reference, not by copy. When called later, it reads their current state (including updated `.grad` fields).

Sticky Point

5. Zero grad before backward.

`p.grad = 0` for all parameters before every `loss.backward()`. Without this, gradients accumulate across training steps.

Sticky Point

6. Topological order is mandatory.

Process a node’s backward only after all nodes that *depend on it* have been processed. `build_topo` guarantees this.

Sticky Point**7. + distributes gradient.**

$c = a + b$: both a and b receive `c.grad` unchanged. A plus node fans out the upstream gradient to all its inputs equally.

Sticky Point**8. $\times$ swaps and scales.**

$c = a \times b$: `a.grad += b.data * c.grad`, `b.grad += a.data * c.grad`. Each input's gradient is scaled by the other's forward value.

Sticky Point**9. Division is power rule.**

$a / b \rightarrow a * b^{-1} \rightarrow$ power rule on b . No new backward logic.

Sticky Point**10. ReLU gates gradient.**

`(out.data > 0) * out.grad`: gradient flows if forward output was positive, blocked if zero or negative.

Sticky Point**11. Seed `loss.grad = 1`.**

$\partial L / \partial L = 1$ always. This is the base case. Everything else is derived from it via the chain rule.

Sticky Point**12. Minus sign in update.**

`p.data -= lr * p.grad`. Gradient points uphill; subtract to go downhill and minimize the loss.

Sticky Point**13. `r*` methods are essential.**

Without `__radd__`, `__rmul__`, etc., expressions like `1 - loss` and `2 * weight` crash. They reverse operand order when Python's normal dispatch fails.

Sticky Point**14. `sum(..., self.b)` trick.**

Starts accumulation from a `Value` (the bias), keeping all arithmetic in the `Value` graph. Starting from `0` (int) would break the first `Value +` call.

Sticky Point**15. `nonlin=False` for last layer.**

Output neurons produce raw values. ReLU on the output would clamp negatives to zero and destroy gradient signal into the loss.

Sticky Point**16. `sz = [nin] + nouts`.**Prepend input dim to get full size list. Consecutive pairs (`sz[i]`, `sz[i+1]`) define each layer's shape.**Sticky Point****17. Abstraction level is flexible.**

Any differentiable function can be a node. Fine-grained (many small ops) and coarse-grained (one op with a known derivative) give identical gradients.

Sticky Point**18. PyTorch is micrograd + efficiency.**Tensors = batches of scalars. GPU kernels = fast matmul. `requires_grad` = selective tracking. The math is identical.

12. Reading Code as a Mental Simulation

Developing strong code-writing intuition means being able to mentally simulate execution. For autograd code specifically, practice the following on any expression you write:

12.1 The Forward-Pass Trace

Given any expression, identify: which `Value` objects exist, what their `.data` values are, what their `._prev` sets contain, and what closure is stored in each `._backward`.

Example: trace `c = a + b` where `a = Value(2.0)` and `b = Value(-3.0)`.

- `c.data = -1.0`
- `c._prev = {a, b}`
- `c._op = '+'`
- `c._backward` is a closure capturing `self=a`, `other=b`, `out=c`
- When called: `a.grad += c.grad`, `b.grad += c.grad`

12.2 The Backward-Pass Trace

Starting from `c.grad = 1`, call `c._backward()`. Deposit 1 into both `a.grad` and `b.grad`. Both are leaves (no `_prev`), so their `_backward` is the no-op lambda. Backward terminates.

For a deeper expression, always work backwards from the output node, calling each closure in topological-reverse order. Draw the graph if needed and annotate `.grad` at each step.

12.3 Identifying Bugs by Pattern

- **Loss explodes or oscillates:** learning rate too high, or zero-grad missing
- **Loss decreases too slowly or stalls:** learning rate too low, or sign of update reversed
- **Gradients are double what expected:** zero-grad not called, old gradients accumulating
- **Node used twice gives wrong gradient:** `=` instead of `+=` in `_backward`
- **TypeError on `2 * a`:** missing `__rmul__`
- **`sum()` returns float not `Value`:** start parameter of `sum` is an int, not `self.b`

12.4 Writing New Operations

Follow the template exactly:

```

1 def your_op(self, other_args):
2     # 1. Compute forward value
3     result_data = ...
4
5     # 2. Create output node
6     out = Value(result_data, (self,), 'your_op_name')
7
8     # 3. Define backward closure
9     def _backward():
10        # local derivative of your_op w.r.t. self
11        local_grad = ...
12        self.grad += local_grad * out.grad    # always +=, never =
13
14    # 4. Attach and return
15    out._backward = _backward
16    return out

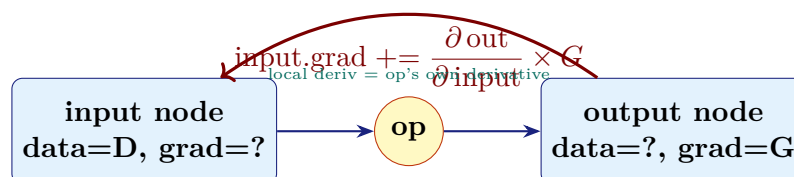
```

The only part that changes between operations is `local_grad`. That is the derivative of your operation with respect to its input, evaluated at the forward value. Everything else is boilerplate.

13. Mental Model — The Three Questions

At any node in the graph, backpropagation answers three questions

1. **What did I output?** — stored in `.data`, computed during the forward pass
 2. **How much does the final loss change when my output changes?** — stored in `.grad`, deposited by the node downstream of me
 3. **How much does my output change when each of my inputs changes?** — the local derivative, computed analytically based on my operation type
- Multiply answers 2 and 3 → deposit into each input's `.grad` using `+=`. Done.



Why 100 lines is enough

The forward pass is just Python arithmetic on `Value` objects — the graph builds itself. The backward pass is one call: `loss.backward()` sorts the graph and runs all the closures. The update is one loop: subtract `lr * grad` from each parameter. Everything else in a production library — tensor operations, GPU kernels, memory management, batching, distributed training — is efficiency. The underlying mathematics is exactly what these 100 lines encode.

Notes by Vatsav Reddy · Under Prof. Praveen Parchuri · Reference: A. Karpathy's "Building Micrograd" lecture · Micrograd source: github.com/karpathy/micrograd